

ML-KEM-768 Stack Optimization Report

=====

Project

Repository: /home/sagar8022/0418/mp22/PQClean

Target implementation: crypto_kem/ml-kem-768/clean

Measurement harness: stack_watermark.c (runtime stack watermark method)

Scope

This document summarizes plain (baseline) and three optimization methods:

- 1) Method-1 (lazy public-key unpack in encryption)
- 2) Method-2 (row-wise matrix generation in keypair)
- 3) Method-3 (row-wise transposed matrix generation in encryption)

Benchmark Method

All numbers were measured using stack_watermark.c with the same compile/link flow for plain and each method branch. Reported values are crypto-only stack usage bytes (after subtracting thread baseline watermark).

Baseline (Plain)

Branch: plain

Result:

- keypair: 6863 bytes
- encaps : 9967 bytes
- decaps : 11071 bytes

Method-1

Branch: feat/kyber-lazy-unpack

Optimization technique:

- Removed full public-key polyvec unpack from encryption path.
- In indcpa_enc, public key polynomials are loaded from bytes one-at-a-time and consumed immediately in the dot product accumulation.
- This avoids keeping the entire unpacked public-key vector live on stack.

Code changes:

- Changed file: crypto_kem/ml-kem-768/clean/indcpa.c
- Function focus: PQCLEAN_MLKEM768_CLEAN_indcpa_enc
- Main edits:
 - Removed helper usage for full unpack into pkpv.
 - Added per-index poly_frombytes + basemul + accumulate flow for v.

Measured result:

- keypair: 6863 bytes
- encaps : 9471 bytes
- decaps : 10575 bytes

Delta vs plain:

- keypair: +0 bytes
- encaps : -496 bytes
- decaps : -496 bytes

Method-2

Branch: feat/kyber-rowwise-keygen

Optimization technique:

- Replaced full matrix A generation/storage during keypair with row-wise generation.

- Only one matrix row is generated and consumed at a time in keypair multiplication.
- This reduces peak stack by avoiding polyvec a[KYBER_K] live storage.

Code changes:

- Changed file: crypto_kem/ml-kem-768/clean/indcpa.c
- Function focus: PQCLEAN_MLKEM768_CLEAN_indcpa_keypair_derand
- Main edits:
 - Added helper: gen_matrix_row(...)
 - Removed full gen_a(a, publicseed)
 - Replaced a[KYBER_K] usage with per-row generation and immediate multiply.

Measured result:

- keypair: 3679 bytes
- encaps : 9967 bytes
- decaps : 11071 bytes

Delta vs plain:

- keypair: -3184 bytes
- encaps : +0 bytes
- decaps : +0 bytes

Method-3

Branch: feat/kyber-rowwise-enc

Optimization technique:

- Replaced full transposed matrix A^T generation/storage in encryption with row-wise generation.
- Only one transposed row is generated and consumed at a time in encryption multiplication.
- This reduces peak stack by removing at[KYBER_K] lifetime from indcpa_enc.

Code changes:

- Changed file: crypto_kem/ml-kem-768/clean/indcpa.c
- Function focus: PQCLEAN_MLKEM768_CLEAN_indcpa_enc
- Main edits:
 - Added helper: gen_matrix_row(...)
 - Removed full gen_at(at, seed)
 - Replaced at[KYBER_K] usage with per-row generation inside the loop.

Measured result:

- keypair: 6863 bytes
- encaps : 6847 bytes
- decaps : 7951 bytes

Delta vs plain:

- keypair: +0 bytes
- encaps : -3120 bytes
- decaps : -3120 bytes

Summary Table

Metric (bytes)	Plain	Method-1	Method-2	Method-3
-----	-----	-----	-----	-----
Keypair	6863	6863	3679	6863
Encaps	9967	9471	9967	6847
Decaps	11071	10575	11071	7951

Best improvement by operation

- Keypair: Method-2 (-3184 bytes)
- Encapsulation: Method-3 (-3120 bytes)
- Decapsulation: Method-3 (-3120 bytes)

Changed Code File(s)

All three methods were implemented by modifying the same source file on different branches:

- crypto_kem/ml-kem-768/clean/indcpa.c

End of report.